

IT-Security

Chapter 5: Application Security



1. Motivation
2. OWASP
3. Threats
4. Solutions for application security
5. Cloud Security
6. IoT Security



What do we want to learn?



- ▶ What are the most common vulnerabilities of (web) applications?
- ▶ What are the OWASP Top 10 vulnerabilities?
- ▶ What measures can I take against attacks?
- ▶ How can I integrate security into software development from the very beginning?



Chapter 5.1: Motivation



Motivation



Common errors

- ▶ an application is planned, developed and tested without considering security
- ▶ many security vulnerabilities are due to poor, careless and insecure programming
- ▶ security technologies are built in (as an alibi) without considering the real threats
- ▶ security is added later as an extension



Example: Apple's SSL bug: goto fail; (2014)

Faulty method:

```
SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa, SSLBuffer signedParams,  
                                uint8_t *signature, UInt16 signatureLen)  
{  
    // ...  
}
```

faulty code snippet:

```
if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)  
    goto fail;  
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)  
    goto fail;  
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)  
    goto fail;  
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)  
    goto fail;  
goto fail;  
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)  
    goto fail;  
  
err = sslRawVerify(ctx,  
                  ctx->peerPubKey,  
                  dataToSign,  
                  dataToSignLen,  
                  signature,  
                  signatureLen);  
  
if(err) {  
    sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "  
               "returned %d\n", (int)err);  
    goto fail;  
}  
  
fail:  
    SSLFreeBuffer(&signedHashes);  
    SSLFreeBuffer(&hashCtx);  
    return err;
```

always
goto fail;

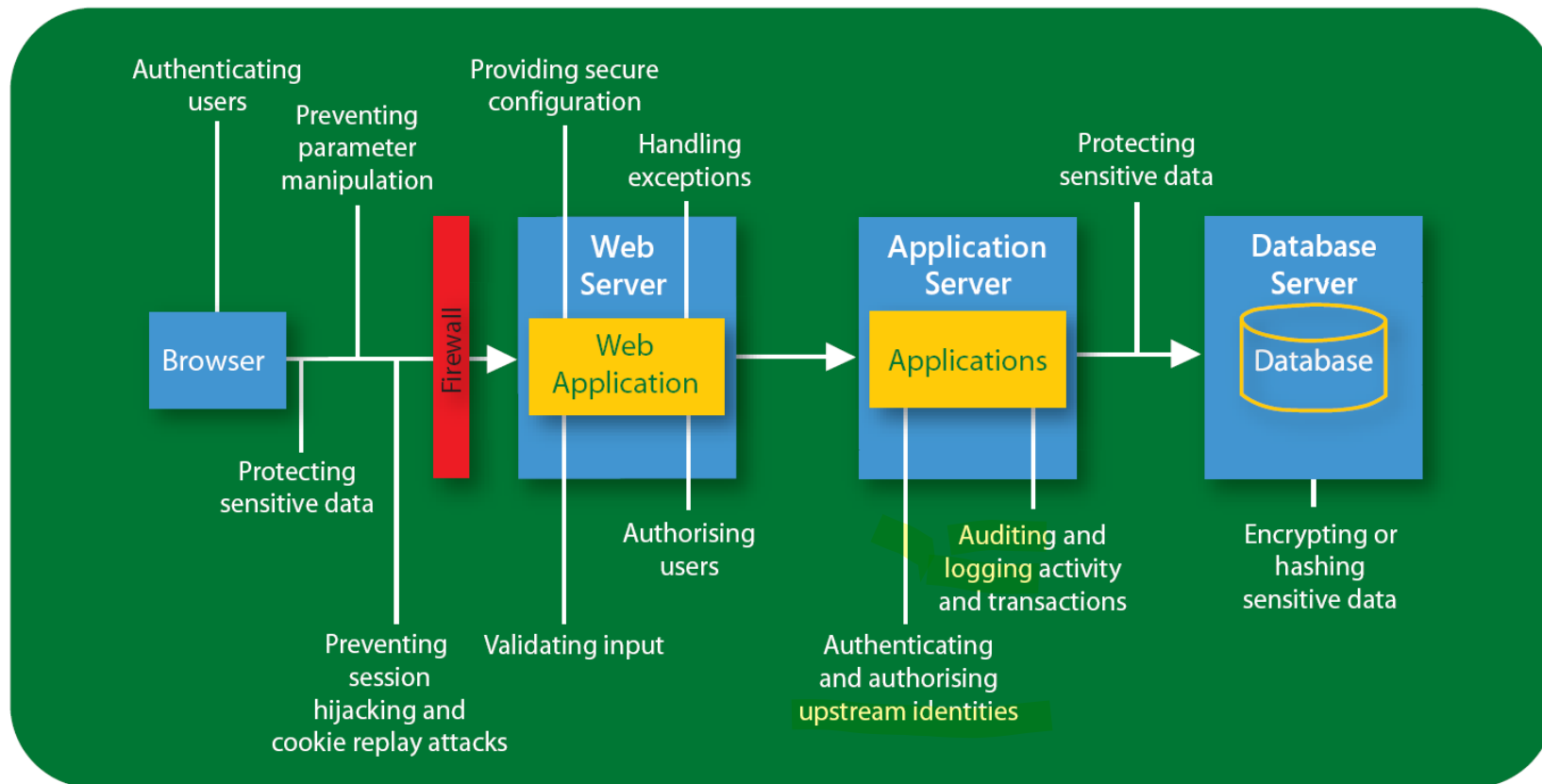
signature check at server key exchange

Always successful?
Not what you would expect!

never called



Top Web Application Security Issues



From „Developer Highway Code (The drive für safer coding)“, Microsoft

<http://download.microsoft.com/documents/uk/msdn/security/The%20Developer%20Highway%20Code.pdf>

upstream identities: combination of user and service

previous slide: what can happen if any of the above code is missing

param manipulation: manipulation of request from browser to web that is not caught
> implement secure checksum, encrypt content (makes it harder to manipulate)

- validate input: prevent injection problems

upstream identities, e.g. user wants to access savings and makes request to application server who takes on an upstream identity with the needed authorizations to complete that request (in this case looking at and manipulating the saving amount)

Chapter 5.2: OWASP

OWAS top 10

insecure design

injection: can insert meta data as input > interpretation of code

- dont send error messages back to user

- statement preparation, adds degree of separation by taking user input as param input

broken access control: person without authorization

■ Broken Access Control

■ Injection

■ Authentication Failures



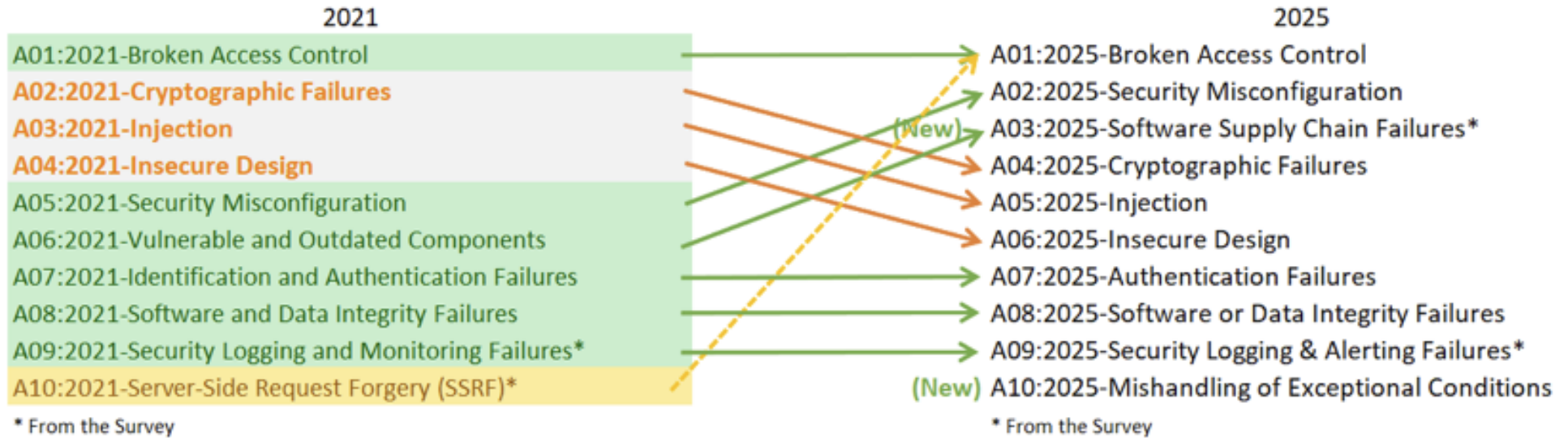
Top Ten Vulnerabilities in Web Applications

List from the Open Web Application Security Project (OWASP)

<https://owasp.org/www-project-top-ten/>



TOP10





OWASP Nr. 1: Broken Access Control



▶ Problem

- ▶ Access control enforces a policy such that users cannot act outside of their intended permissions.
- ▶ Failures typically lead to unauthorized information disclosure, modification, or destruction of all data

▶ Typical access control vulnerabilities

- ▶ Metadata manipulation (such as JWT)
- ▶ APIs with missing access controls for POST, PUT, DELETE

▶ Elevation of Privileges

- ▶ Vulnerabilities that allow an attacker to extend his privileges
- ▶ **Horizontal**: access to resources of other users of the same privilege level
- ▶ **Vertical**: access to resources of a higher privilege level (e.g., admin)



More access control vulnerabilities

▶ Cross-Origin Access

- ▶ problem: Microservices and REST APIs often need cross-domain accesses
- ▶ measure: **SOP Same Origin Policy**
- ▶ **CORS (Cross Origin Resource Sharing)**: Access for JavaScript to other origins controlled via http-response header

▶ Direct object reference

- ▶ Access to objects via modification of URL (e.g., change invoice ID).
- ▶ Measures: perform access control before granting access, no direct object references in URL, indirect object references are more secure.

▶ Path Traversal / Force browsing

- ▶ Direct access to files of the web server
- ▶ Cause: insufficient validation
- ▶ Measures: Indirect access to filename, whitelist validation, normalization.

manipulation of URL params allows access to different data even if not authorized to view

How to prevent Broken Access Control?

- ▶ The main measure is to **enforce access control** before delivering any data or function
- ▶ Design your application according to **secure access control patterns** (least privilege, need to know, deny by default, ...)
- ▶ There are many measures to secure access control
 - ▶ minimize CORS usage
 - ▶ disable web server directory listing
 - ▶ rate limit APIs to protect against attack tools
 - ▶ log access control failures and alert admins
 - ▶ perform unit and integration tests for access control



OWASP Nr. 5: Injection



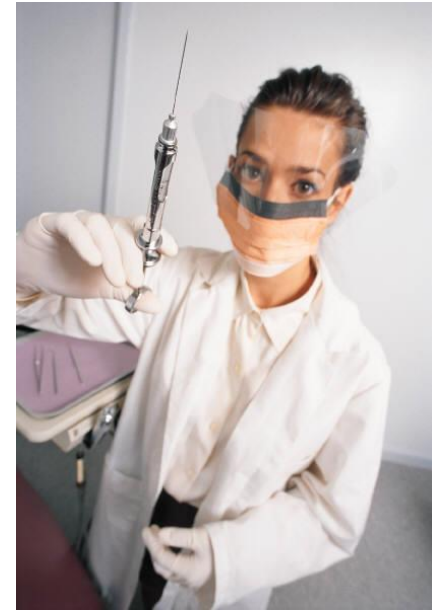
- ▶ An Application uses code interpreted at runtime
 - ▶ interpreted code is dynamically created or modified at runtime
 - ▶ user input flows directly into created code
 - ▶ examples: SQL, Shell, LDAP, XML, ...

- ▶ Protective measures against attacks
 - ▶ **Input Sanitization**: escape or remove the metacharacters
 - ▶ **Blacklisting** : only unwanted characters in input are escaped or removed
 - ▶ **Whitelisting** : only input with allowed characters will be passed through
 - ▶ use **frameworks** that take over protection
e.g. LdapQueryBuilder in Spring-LDAP
 - ▶ use **Web Application Firewalls (WAF)**



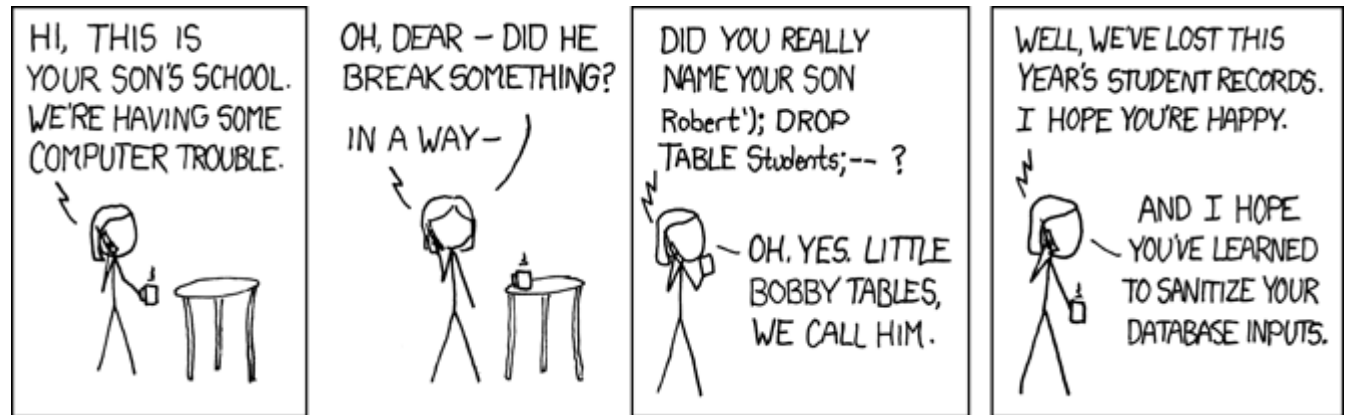
SQL Injection

- ▶ Problem
 - ▶ Dynamically created database queries are manipulated by user input
- ▶ Possible damage
 - ▶ disclosure of confidential data
 - ▶ malicious modification and deletion of data
 - ▶ injecting foreign code to damage systems
 - ▶ bypassing password protection of web applications
- ▶ Occurs "only" in applications that work with SQL-based databases





Examples for SQL Injection



<http://xkcd.com/327/>

```
SELECT * FROM Usr WHERE UserName = 'john' – ' AND Password= ''
```

evals and before or

```
SELECT * FROM Usr WHERE UserName = 'john' OR 'a' = 'b' AND Password= ''
```

true

false

```
SELECT * FROM Customer WHERE CustId = 1; DELETE FROM Customer
```

next statement

```
SELECT Id, Title, Abstract FROM News  
WHERE Category= 1 UNION SELECT 1, UserName, Passwd FROM Usr
```



SQL Injection: provoke error messages to get internal knowledge of the database

`http://www.stock.example/fund.asp?id=1+OR+qwe=1`

Error message:
Invalid Column name qwe

`SELECT * FROM news WHERE id = 1 HAVING 1=1`

Error message:
Attribute news.id must be GROUPED or used in an aggregate function



Solutions for SQL Injection (1)

- ▶ Use secure APIs
 - ▶ prevent direct access to SQL interpreter
 - ▶ offer parameterized interfaces
- ▶ Prepared statements (parameterized queries)
 - ▶ perform automatic escaping of metacharacters
 - ▶ additionally have better performance for DB queries
- ▶ Example for a prepared statement with JDBC (Java)

```
PreparedStatement ps = conn.prepareStatement("UPDATE news SET  
title=? WHERE id = ?;  
...  
ps.setString(1, title);  
ps.setInt(2, id);    id is user input and param input  
int rowCount = ps.executeUpdate( );
```




Solutions for SQL Injection (2)

- ▶ Administrative measures
 - ▶ minimal rights for SQL users
 - ▶ no system accounts for database users
 - ▶ install a Web Application Firewall
 - ▶ whitelist input validation
- ▶ Neutralize SQL metacharacters ("data washing")
 - ▶ Important: identify all metacharacters and deactivate them
 - ▶ Examples
 - ▶ double single quotes
 - ▶ double backslash
 - ▶ encapsulate resulting strings in new quotes
 - ▶ apply functions that validate numeric input
- ▶ Example: OWASP Enterprise Security API (<http://www.owasp.org/index.php/ESAPI>)
 - ▶ API with databank encoder for some databases in different programming languages



Similar problems

▶ Shell command injection (OS command injection)

- ▶ OS commands are called from a web application with dynamic parameters
- ▶ allows an attacker to execute arbitrary OS commands on the server and to compromise applications and all its data and part of the hosting infrastructure

▶ Example in Perl

```
$username = $form{"username"};  user input  
print 'finger $username';        shell cmd
```

Call: finger john; rm -rf /

metacharacter escapes current statement and executes new one

▶ Solutions:

- ▶ try to get along without shell commands
- ▶ call external programs directly (system ,exec)
- ▶ programming the functionality or use secure APIs (e.g. mail)
- ▶ handle metacharacters
- ▶ prevent user input in command arguments

▶ OWASP Nr. 5: Cross-Site-Scripting (XSS), since 2021 classified as injection problem



▶ Problem

- ▶ HTML constructs injected by an attacker are passed to other users' browsers via a web application
- ▶ JavaScript is automatically executed in the browser
- ▶ user input to a web page is passed unfiltered as a query string and may contain malicious tags
- ▶ XSS is a metacharacter and an output problem

intention is to place a script that is then being executed by another user (e.g. read cookies from browser and send them back)

▶ Possible consequences:

- ▶ running malicious script in a client's browser
- ▶ stealing users' cookies and breaking authentication

e.g. web shop, place javascript into backend of comment so that when someone clicks on it, it will execute the hidden code

▶ Occurs especially in web applications where users provide the content

- ▶ web frontends for e-mail systems and newsgroups
- ▶ web-based discussion forums
- ▶ social networks
- ▶ can be combined with "social engineering"

▶ Types of XSS

- ▶ Classic classification (2005)
 - ▶ **Persistent (Stored) XSS**: the application stores unchecked user input
 - ▶ **Reflected XSS**: unchecked user input ends up directly in the HTML output again
 - ▶ **DOM Based XSS**: source of data is in the DOM, data ends up in the DOM again
- ▶ Modern classification (2012)

AJAX
HTML5

Where untrusted data is used			Data Persistence	HTML5
XSS	Server	Client		
Stored DB	Stored Server XSS	Stored Client XSS		
Reflected	Reflected Server XSS	Reflected Client XSS		

- ❑ DOM-Based XSS is a subset of Client XSS (where the data source is from the client only)
- ❑ Stored vs. Reflected only affects the likelihood of successful attack, not nature of vulnerability or defense

https://owasp.org/www-community/Types_of_Cross-Site_Scripting



Measures against XSS

▶ Server XSS Defense

- ▶ perform context-sensitive output encoding on the server
- ▶ HTML encoding of untrusted data before inserting it into HTML output (HTML body, attributes, JavaScript, CSS, URL).
 - remove script, transform to string
- ▶ But pure **encoding** is usually not enough, because data between <script> tags, in event handlers, in CSS or in a URL can still contain XSS attacks
 - ▶ therefore, use security encoding libraries (e.g. Microsoft Anti-Cross Site Scripting Library, OWASP Java Encoder Project)
 - ▶ use frameworks that mask XSS by design (e.g. Ruby on Rails, React JS)
 - ▶ separate untrusted data from active browser content

▶ Client XSS Defense

- ▶ use secure JavaScript APIs

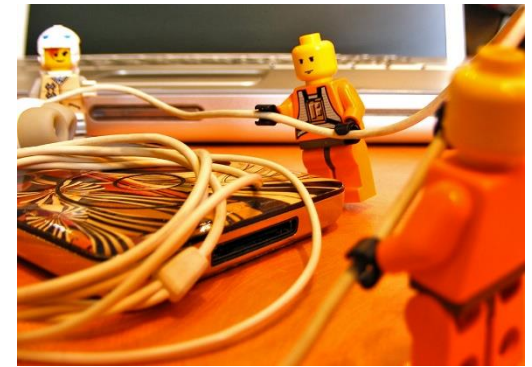


OWASP Top 7: Authentication Failures



- ▶ **Broken Authentication**
 - ▶ application functions related to authentication and session management are often implemented incorrectly
 - ▶ this allows attackers to compromise passwords or session tokens
 - ▶ this allows them to assume the identity of other users temporarily or permanently
- ▶ **Session management** is the basis for authentication and access control in stateful applications
- ▶ **Session-Hijacking**
 - ▶ an attacker has access to a victim's session ID and thus bypasses authentication

▶ Measures against gegen Session-Hijacking



- ▶ **Keep session ID secret (primary measure !!)**
- ▶ **use a secure transmission channel (encrypt, e.g. https)**
- ▶ limit lifetime of session (**time out**)
- ▶ generate a new random session ID when user logs in
- ▶ do not use persistent sessions
- ▶ do not allow parallel sessions of a user (no concurrent sessions)
- ▶ use **Session Binding**: Bind session ID to client IP address or browser fingerprint
- ▶ encrypt content of authentication cookies
- ▶ set cookie attributes (secure, HttpOnly, domain, expires)
- ▶ use an Anti-CSRF token (**Synchronizer token** provides protection against replay attacks)
- ▶ include a logout option in your application

secure: only https

HttpOnly: not readable with javascriptdomain: fix the serverexpires: time out



Chapter 5.3: Other threats



Other threats to web applications

- ▶ Attacks occurs at various levels of web applications
 - ▶ UI
 - ▶ Network
 - ▶ Server
- ▶ **Clickjacking (UI)**
 - ▶ is when an attacker uses multiple transparent or opaque layers to trick a user into clicking an invisible action
 - ▶ Countermeasure: **Frame Busting** prevents framing (set X-Frame-Options: DENY/SAMEORIGIN)
 - ▶ **CSP Content Security Policy:** declare in HTTP response header which dynamic resources, such as JavaScript, CSS, the browser is allowed to load
e.g. do not allow JavaScript inline but only in files, content only type text/json
<https://content-security-policy.com>



more threats to web applications

▶ **Replay-Attacks (network)**

- ▶ replay request by an attacker or accidentally by a user
- ▶ Measure: Anti-Replay Token, Anti-CSRF Token, Synchronizer Token

cross site request forgery

▶ **Remote Code Execution (RCE)** real problem

- ▶ attackers can run code on the target machines via a vulnerability
- ▶ e.g. command injection, path traversal and execution of files, exploitation of vulnerabilities in the network or IT systems
- ▶ RCE is absolute worst case!
- ▶ Measures: Defense in Depth, Updating all Components and libraries, Sandboxing, Input Sanitization

OWASP 5 Injection
8 Integrity failures
3 supply chain failures



DOS Denial of Service



- ▶ DOS cannot be prevented
- ▶ An **emergency plan** enables you to react appropriate in the event of a DoS and reduces the damage
- ▶ There are different variants of DOS: Application crash, OS crash, CPU, memory or resource overload.
- ▶ Measures to reduce vulnerability
 - ▶ write solid code
 - ▶ perform testing and performance analysis with profiler
 - ▶ perform monitoring to detect changes of application behavior
 - ▶ do not trust any data over the network
 - ▶ allow complex operations only if you are sure that a trusted client is sitting at the other end
 - ▶ request resources as late as possible and release them as early as possible
 - ▶ validate requests before sending error messages and implement a controlled error handling
 - ▶ disconnect clients without authentication when attacked
 - ▶ rate limiting and tarpits
slow down



Chapter 5.4: Solutions for Application Security

- Software development rules
- Input validation
- Error handling
- Logging and monitoring
- Secure administration and operation



Software Development Rules

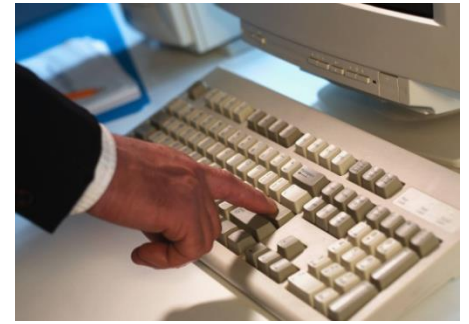
- ▶ Best practices for good quality code
 - ▶ avoid duplicate code
 - ▶ isolate functionality
 - ▶ limit the length of code blocks
 - ▶ limit the functionality of functions
 - ▶ limit the need for comments
 - ▶ limit the scope of variables
 - ▶ perform unit tests

- ▶ Enemies of secure code
 - ▶ ignorance
 - ▶ disorder
 - ▶ deadlines, time pressure





Validation all input



- ▶ Motivation: "Wrong" inputs can cause programs to behave in malicious manner
- ▶ **Canonicalization & Normalization**
 - ▶ reduce to the standardized, simplest, form of representation
- ▶ **Sanitization**
 - ▶ Remove/escape any unsafe characters from user input
- ▶ **Validation**
 - ▶ ensure that data has the expected format and type
 - ▶ check the integrity and correctness of input data, ensuring it meets specified criteria or constraints.
 - ▶ protect input to subsystems with cryptographic check sums (e.g. MAC)



Canonicalization



- ▶ Problem:
 - ▶ there are often multiple options when assigning a name
 - ▶ attackers can exploit code that makes decisions based on filenames or URLs
- ▶ Examples of problems with URLs:
 - ▶ ROSE == R%6fse ??? %2d > . one standard
 - ▶ <http://www%2emicrosoft%2ecom%2ftechnet%2fsecurity>
 - ▶ <http://172.43.122.12> == <http://2888530444>
 - ▶ Dotless-IP: $a.b.c.d = (a * 256^3) + (b * 256^2) + (c * 256^1) + (d * 256^0)$
- ▶ Examples of potential file name problems:
 - ▶ old file name formats (Fiscal04Budget.xls == FISCAL~1.xls)
 - ▶ dot at the end of filename allowed, but will be removed automatically
 - ▶ absolute and relative file names
 - ▶ upper and lower case
- ▶ Measure
 - ▶ canonicalize filenames (long filenames only, remove dot at end, absolute path) and URLs



Validation

- ▶ Make sure that all inputs are identified and validated
 - ▶ validate before any other task
 - ▶ perform authorization together with validation
- ▶ Write functions that perform validation
- ▶ Check length, range (e.g. numeric and non-negative) , format
- ▶ Identify all sources of input to a web application (URL parameters, http header, cookies, text inputs, checkboxes, radio buttons, drop-down lists, buttons, hidden fields, ...)
WARNING: even predefined values can be manipulated
- ▶ Use whitelisting instead of blacklisting for filtering
 - ▶ **Whitelisting** only allows data that is believed to be harmless
 - ▶ **Blacklisting** allows everything that is not explicitly forbidden
- ▶ Never use client-side validation as the only basis for decision-making

check

- origin
- size
- lexical content
- syntax
- semantic





Handle meta character proper

- ▶ Meta characters
 - ▶ Are characters that do not stand for themselves within a certain context but have a special meaning.
 - ▶ when passing data to a subsystem they change from a text character to a control character
- ▶ Measures
 - ▶ insert escape character like /, if meta character also makes sense as normal character
 - ▶ otherwise: remove metacharacters
 - ▶ use subsystems to interpret metacharacters (e.g. prepared statements, DOM)
 - ▶ use encoding (e.g. HTML encoding)
 - ▶ encapsulate the communication with other systems
 - ▶ keep permissions in subsystems minimal



Error Handling



- ▶ Clean and stable error handling is important for secure software
 - ▶ secure software can slow down, but it should not crash or give incorrect results.
 - ▶ avoid that a user gets more rights by an error
 - ▶ after an error, an application must be brought to a stable and clean state.
- ▶ Do not correct an invalid input to make it a valid one
- ▶ Logging of errors
 - ▶ the system logs are usually not sufficient (web server, database, firewall, ...)
 - ▶ additional logging on application level is necessary to find out where the cause of the error lies



Rules for Exception Handling



- ▶ If you find a security related error
 - ▶ correct the bug and look for similar problems in other code parts
 - ▶ correct it as close to the vulnerability as possible
 - ▶ treat the cause not the symptoms
 - ▶ extend the test drivers to include the bug

- ▶ Do not send detailed error messages to the client
 - ▶ no database errors, access violations, file names, table names
 - ▶ because otherwise the attacker gets information about internal structures of the application
 - ▶ catch all errors and show only a general error message page
 - ▶ write detailed error description in the log files
 - ▶ generate log files on application level

Information concealment is the first line of defense



Anti Pattern: error output to the user

Information disclosure and leakage

HTTP Status 500 - An exception occurred processing JSP page /basket.jsp at line 244

type Exception report

message An exception occurred processing JSP page /basket.jsp at line 244

description The server encountered an internal error that prevented it from fulfilling this request

exception

org.apache.jasper.JasperException: An exception occurred processing JSP page /basket.jsp at line 244

```

241:                stmt.execute();
242:                stmt.close();
243:            } else {
244:                stmt = conn.prepareStatement("UPDATE BasketContents SET quantity = " + Integer.parseInt(value) + " WHERE basketid=" + basketId +
245:                " AND productid = " + prodId);
246:                stmt.execute();
247:                if (Integer.parseInt(value) < 0) {

```

Stacktrace:

```

org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:568)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:455)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:395)
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:339)
javax.servlet.http.HttpServlet.service(HttpServlet.java:727)
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:52)

```

root cause

```

javax.servlet.ServletException: java.sql.SQLException: Unexpected token: % in statement [UPDATE BasketContents SET quantity = 4 WHERE basketid=%27 AND productid = 20]
org.apache.jasper.runtime.PageContextImpl.doHandlePageException(PageContextImpl.java:916)
org.apache.jasper.runtime.PageContextImpl.handlePageException(PageContextImpl.java:845)
org.apache.jsp.basket_jsp._jspService(basket_jsp.java:390)
org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
javax.servlet.http.HttpServlet.service(HttpServlet.java:727)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:432)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:395)

```

note The full stack trace of the root cause is available in the Apache Tomcat/7.0.2 logs.

Apache Tomcat/7.0.2

```

org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:52)

```

note The full stack trace of the root cause is available in the Apache Tomcat/7.0.2 logs.

Apache Tomcat/7.0.2



Is this a problem?

- ▶ Step 1: start **Google search** for "apache tomcat 7.0.2 vulnerabilities".

select the hit <https://tomcat.apache.org/security-7.html> and you find a lot of vulnerabilities and the version where they are fixed, for example

- ▶ Authentication weakness [CVE-2021-30640](#)
Queries made by the JNDI Realm did not always correctly escape parameters. Parameter values could be sourced from user provided data (eg user names) as well as configuration data provided by an administrator.

- ▶ Step 2: Go to the **NIST National Vulnerability Database**
<https://nvd.nist.gov/vuln/search> and search for CVEs for apache tomcat version 7.0.2

You find many vulnerabilities, among them, for example:

- ▶ <https://nvd.nist.gov/vuln/detail/CVE-2018-1336>
An improper handling of overflow in the UTF-8 decoder with supplementary characters can lead to an infinite loop in the decoder causing a Denial of Service.



Risks always remains, therefore logging is necessary

OWASP Nr. 9: Security Logging & Alerting Failures

- ▶ A residual risk always remains: For this reason, attacks must not only be defended against, but also detected.
- ▶ The most important source of information for detecting attacks and intrusions are logs.

Good logging is the basis for forensic
- ▶ Central Questions:
 - ▶ What will be logged? (https://www.owasp.org/index.php/Logging_Cheat_Sheet)
 - ▶ What will be not logged?
 - ▶ How are the logs collected? (e.g. logstash, ...)
 - ▶ How are the logs analyzed and visualized? (e.g. Kibana, ...)
- ▶ There are ready-made tool chains for log analysis (e.g. ELK Stack <https://www.elastic.co>)



Rules for auditing and logging and monitoring and alerting



▶ Auditing

- ▶ Recording of important **user activities** (**traceability**)

▶ Logging

- ▶ Log all **security relevant events**

▶ Monitoring & Alerting

- ▶ Collect log data to **observe the performance** of an application, detect abnormal behaviour and **trigger alerts**

▶ Rules

- ▶ Identify malicious behavior
- ▶ Know the normal behavior of the application (good traffic)
- ▶ Audit and log activities on all application layers
- ▶ Restrict access to log files and protect log integrity
- ▶ Do not include private or sensitive data (e.g. passwords) in log files
- ▶ Secure and analyze log files regularly
- ▶ Integrate Logging with Monitoring



Secure administration and operation



- ▶ In addition to secure programming, secure operation is also required to reduce the attack vectors
 - ▶ establish a secure infrastructure (FW, WAF, SSL gateway, access gateway, security zones, ...)
 - ▶ separate the production systems from test and development systems (especially the credentials)
 - ▶ secure the remote access for administration especially strong
- ▶ Establish an **Incidence Response and Emergency Management**
- ▶ **Change Management**
 - ▶ Many security vulnerabilities are introduced into the systems by bug fixing and adding new features
 - ▶ Track and test all changes



Secure administration and operation

- ▶ **System Hardening**: reduce the attack surface of all deployed systems by restrictive configuration
 - ▶ change insecure default settings
 - ▶ adjust access rights
 - ▶ disable unnecessary services
 - ▶ perform vulnerability scans

- ▶ **Decommission**
 - ▶ remove authentication and authorization settings (accounts, key, certificates, ...)
 - ▶ reset configuration settings (FW settings, open ports, DNS entries, ...)
 - ▶ delete or save the data

- ▶ **Multi layer defense**
 - ▶ if one security mechanism fails, another should handle the problem
Example: restrictive administration of a DB, in case that SQL input validation fails



Chapter 5.5: Security in cloud platforms



Security in Cloud Platforms



- ▶ In cloud platforms there is the possibility of **Continuous Integration CI** and **Continuous Deployment CD**
- ▶ Agility increases, but so do security risks
- ▶ The areas of development and operation are growing together more and more in modern cloud platforms based on micro services
- ▶ Developers must also deal with administration and operation
→ there is a new role in software engineering **DevOps**
- ▶ Automate as much as possible in cloud platforms
 - ▶ this ensures traceability of what is happening in the platform
 - ▶ it reduces errors and human interaction



DevOps/Cloud Security Issues



- ▶ Securing the **CI/CD pipeline**
 - ▶ Goal: Deployment from trusted sources
 - ▶ Risk: Failures due to untested or insecure code.
 - ▶ Measures: Version control in GitHub, 2FA, 4-eye-principle, automatic tests, security checks, artifact repository, image registry, rollback options
- ▶ Image and container security
 - ▶ Risk: unauthorized changes on the prod environment
 - ▶ Measures:
 - ▶ use secure base images/containers
 - ▶ vulnerability checks for used libraries, external services, APIs, frameworks, configurations
- ▶ Network security
 - ▶ Protection against other applications, clients, attackers on the platform
 - ▶ Isolation
 - ▶ Ingress and Egress control



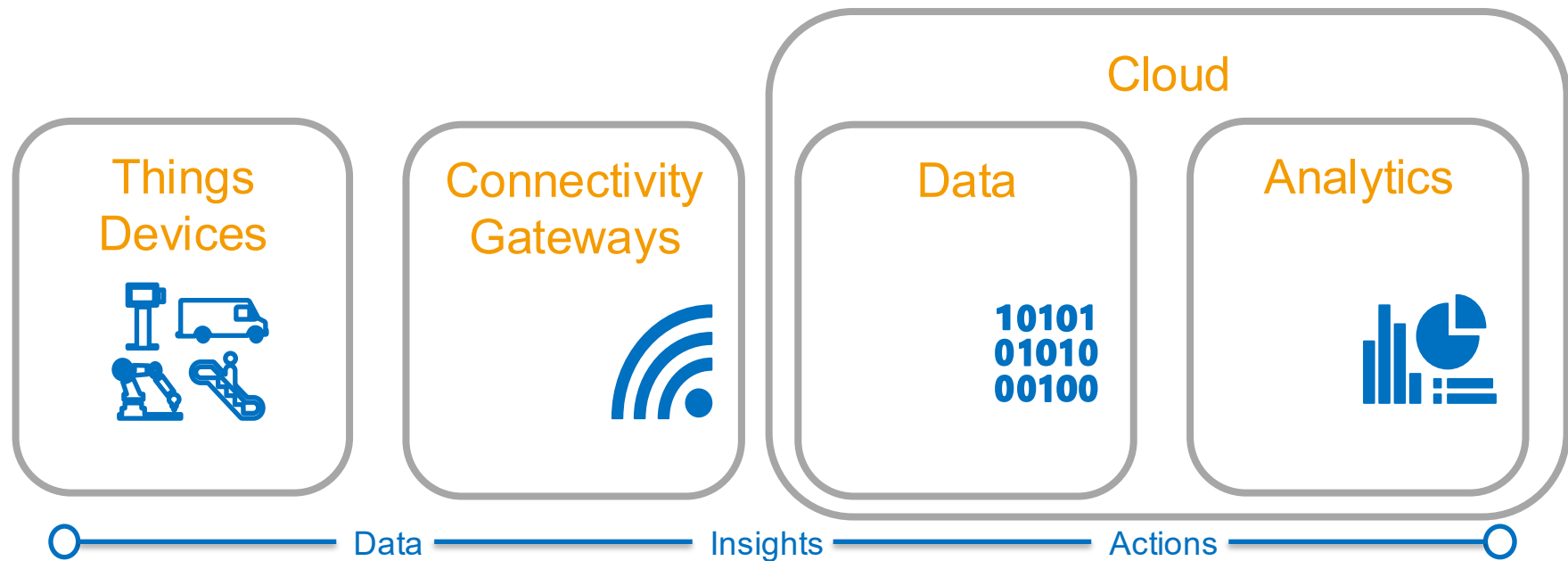
DevOps/Cloud Security Issues

- ▶ Access and Account Management
 - ▶ Control the rights and roles on different levels
 - ▶ Technical accounts (administration of the cloud platform and infrastructure, GitOps Cockpit based on cluster security policies)
 - ▶ Functional accounts (application administration, user management, data services, reporting, billing, hotline)
 - ▶ DevOps/Maintenance Accounts (Error Analysis and Recovery, Logging, Monitoring, Alerting)
- ▶ Secret Management
 - ▶ Goal: passwords, API keys, access tokens must be protected.
 - ▶ Measures: Sealed Secrets, Key Vault, Secret Manager
- ▶ Cluster Backup and Recovery
 - ▶ Prevention of data loss
 - ▶ Fast failover



Chapter 5.6: IoT Security

IoT-Security (Internet of Things)



- ▶ IoT means: many devices, lots of communication, lots of data
- ▶ Examples for IoT applications
 - ▶ Smart Home, Smart Mobility, Smart Factory, Smart Health

IoT brings new challenges for IT security

- ▶ Securing devices that do not have physical access protection
- ▶ Updating (old) devices for long-term operation
- ▶ The generated data enables control of movement and behavior
- ▶ Autonomous systems can make independent decisions
- ▶ Massive communication and sharing of data
- ▶ Many standards without security features
- ▶ Large amount of cheap consumer products

Source: Wendzel S., IT –Sicherheit für TCP/IP- und IoT-Netzwerke, Springer-Vieweg, 2018, eBook in Bibliothek



IoT has special risks



Threats

- ▶ Hijacked Devices (IoT Botnets)
- ▶ information leakage
- ▶ disruption of services, functionality



Vulnerabilities

- ▶ exposed device in WWW (Shodan tool shows exposed devices <https://www.shodan.io/>)
- ▶ insecure web interfaces (bad session management, default credentials)
- ▶ insecure frameworks and protocols
- ▶ generic security vulnerabilities (no access control, no logging, insecure storage, hardcoded passwords)
- ▶ outdated devices



IoT requires more skills than pure software engineering:
systems engineering is additionally required





Security measures for IoT

- ▶ All rules for secure application development apply to IoT.
- ▶ Special focus must be placed on the following measures
 - ▶ encrypt all communication
 - ▶ secure key management
 - ▶ (mutual) authentication mechanism
 - ▶ no execution of unauthorized code
 - ▶ secure updates
 - ▶ code signing mechanism
 - ▶ implement devices as clients (do not host services, do not act as servers)
 - ▶ least privilege
 - ▶ intrusion detection



OWASP for IoT

I1 Weak Guessable, or Hardcoded Passwords

I2 Insecure Network Services

I3 Insecure Ecosystem Interfaces

I4 Lack of Secure Update Mechanism

I5 Use of Insecure or Outdated Components

I6 Insufficient Privacy Protection

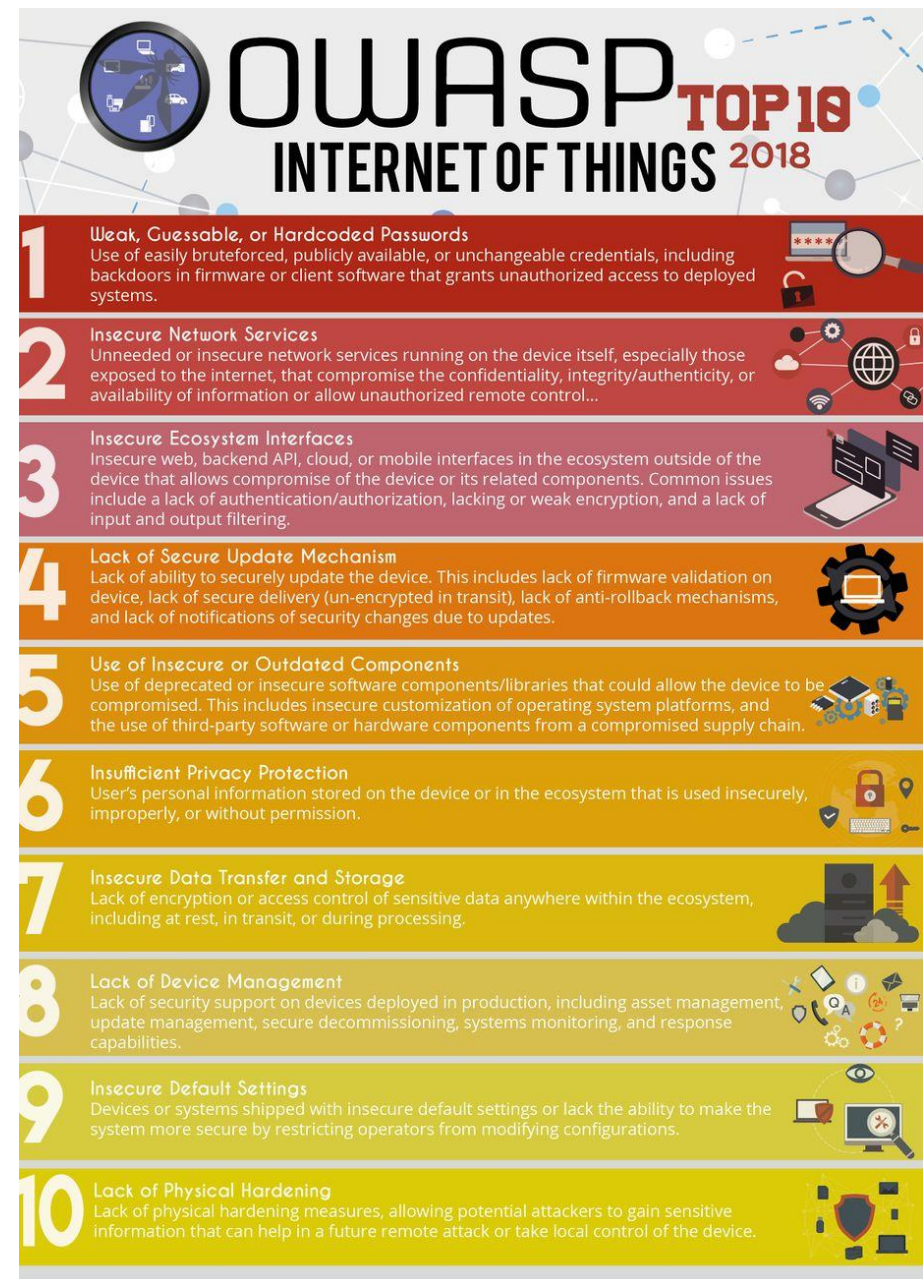
I7 Insecure Data Transfer and Storage

I8 Lack of Device Management

I9 Insecure Default Settings

I10 Lack of Physical Hardening

<https://owasp.org/www-project-internet-of-things/>





Summary application security



- ▶ Every software engineer must know the most important threats and protect his applications against them.
- ▶ OWASP provides a good overview of application vulnerabilities.
- ▶ All input to an application must be validated, canonicalized, filtered, sanitized and escaped.
- ▶ In addition, measures such as secure programming, logging, secure error handling, security testing and secure operation must be implemented.
- ▶ In certain application domains (e.g., cloud, IoT), additional measures must be considered